

API REST & Sécurité — CanBankX

Auteur : Pascal Bourgoïn **Date :** Mars 2026 **Compléments :** [arc42.md](#) [§8.1–8.2](#) · [ADR001–hexagonal](#) · [ADR005–authentication](#)

Table des matières

- 1. [Conventions REST](#)
- 2. [Catalogue des endpoints](#)
- 3. [Sécurité des mots de passe — BCrypt 8](#)
- 4. [Flux MFA en deux étapes](#)
- 5. [Spring Security stateless](#)
- 6. [Filtrage des en-têtes par KrakenD](#)
- 7. [Gestion des erreurs normalisée](#)
- 8. [Stratégie de versionnage](#)
- 9. [CORS](#)

1. Conventions REST

Chaque service expose ses endpoints avec un **préfixe de namespace** qui sert à la fois de version implicite et de discriminant pour le routage KrakenD.

Service	Préfixe interne	Préfixe public (via Gateway)
identity-service	/identityservice/**	/api/clients/**, /api/auth/**
account-service	/accountservice/**	/api/accounts/**
payment-service	/paymentservice/**	/api/transactions/**

Codes HTTP utilisés :

Code	Signification dans CanBankX
200 OK	Lecture réussie ou action sans création de ressource
201 Created	Ressource créée (client, compte, transaction)
400 Bad Request	Échec de validation @Valid (champs manquants ou invalides)
401 Unauthorized	Mauvais mot de passe, token MFA expiré, OTP invalide
404 Not Found	Ressource inexistante (client, compte, transaction)
409 Conflict	Email ou NAS déjà enregistré (contrainte UNIQUE)
422 Unprocessable Entity	Solde insuffisant, compte non actif
500 Internal Server Error	Exception non gérée (capturée par GlobalExceptionHandler)

2. Catalogue des endpoints

2.1 identity-service (port 8081)

Gestion des clients

Méthode	Chemin public	Chemin interne	Description	Auth requis
---------	---------------	----------------	-------------	----------------

Méthode	Chemin public	Chemin interne	Description	Auth requis
POST	/api/clients	/identityservice/clients	Inscription + envoi OTP email	Non
POST	/api/clients/{id}/verify	/identityservice/clients/{id}/verify	Vérification OTP (KYC)	Non
POST	/api/clients/{id}/activate	/identityservice/clients/{id}/activate	Activation admin (bypass OTP)	Non
GET	/api/clients/{id}	/identityservice/clients/{id}	Récupérer un client par ID	Non*
GET	/api/clients	/identityservice/clients	Lister tous les clients	Non*
PATCH	/api/clients/{id}/status	/identityservice/clients/{id}/status	Modifier le statut (ACTIVE / SUSPENDED)	Non*

*Spring Security est configuré en mode permissif pour le contexte académique — tous les endpoints sont publics sur le réseau interne Docker.

Authentification MFA

Méthode	Chemin public	Chemin interne	Corps / Réponse
POST	/api/auth/login	/identityservice/auth/login	{email, password} → {challengeToken}
POST	/api/auth/mfa	/identityservice/auth/mfa	{challengeToken, otpCode} → {message: "SUCCESS"}

2.2 account-service (port 8082)

Méthode	Chemin public	Chemin interne	Description
POST	/api/accounts	/accountservice/accounts	Ouvrir un compte (CHECKING / SAVINGS)
GET	/api/accounts/{id}	/accountservice/accounts/{id}	Récupérer un compte par UUID
GET	/api/accounts/client/{clientId}	/accountservice/accounts/client/{clientId}	Lister les comptes d'un client
GET	/api/accounts/{id}/summary	(agrégé par KrakenD)	Solde + 5 transactions récentes
PATCH	(interne)	/accountservice/accounts/number/{n}/debit	Débiter (appelé par payment-service)

Méthode	Chemin public	Chemin interne	Description
PATCH	(interne)	/accountservice/accounts/number/{n}/credit	Créditer (appelé par payment- service)

L'endpoint `/api/accounts/{id}/summary` n'existe pas en tant que tel sur `account-service`. Il est synthétisé par KrakenD en appelant en parallèle `account-service` et `payment-service`, puis en fusionnant les deux réponses dans un objet JSON avec les groupes `account` et `recentTransactions`.

2.3 payment-service (port 8083)

Méthode	Chemin public	Chemin interne	Description
POST	/api/transactions	/paymentservice/transactions	Soumettre une transaction (DEBIT / CREDIT / TRANSFER)
GET	/api/transactions/{id}	/paymentservice/transactions/{id}	Récupérer une transaction par UUID
GET	/api/transactions/account/{n}	/paymentservice/transactions/account/{n}	Historique d'un compte (5 dernières)

Header obligatoire pour **POST /api/transactions** :

```
Idempotency-Key: <uuid-v4>
```

KrakenD transmet ce header uniquement vers `payment-service`. Il est ignoré sur les autres services.

3. Sécurité des mots de passe — BCrypt 8

Les mots de passe ne sont jamais stockés en clair. Lors de l'inscription (**POST /api/clients**), `ClientService` encode immédiatement le mot de passe avant toute persistance :

```
client.setPasswordHash(passwordEncoder.encode(request.getPassword()));
```

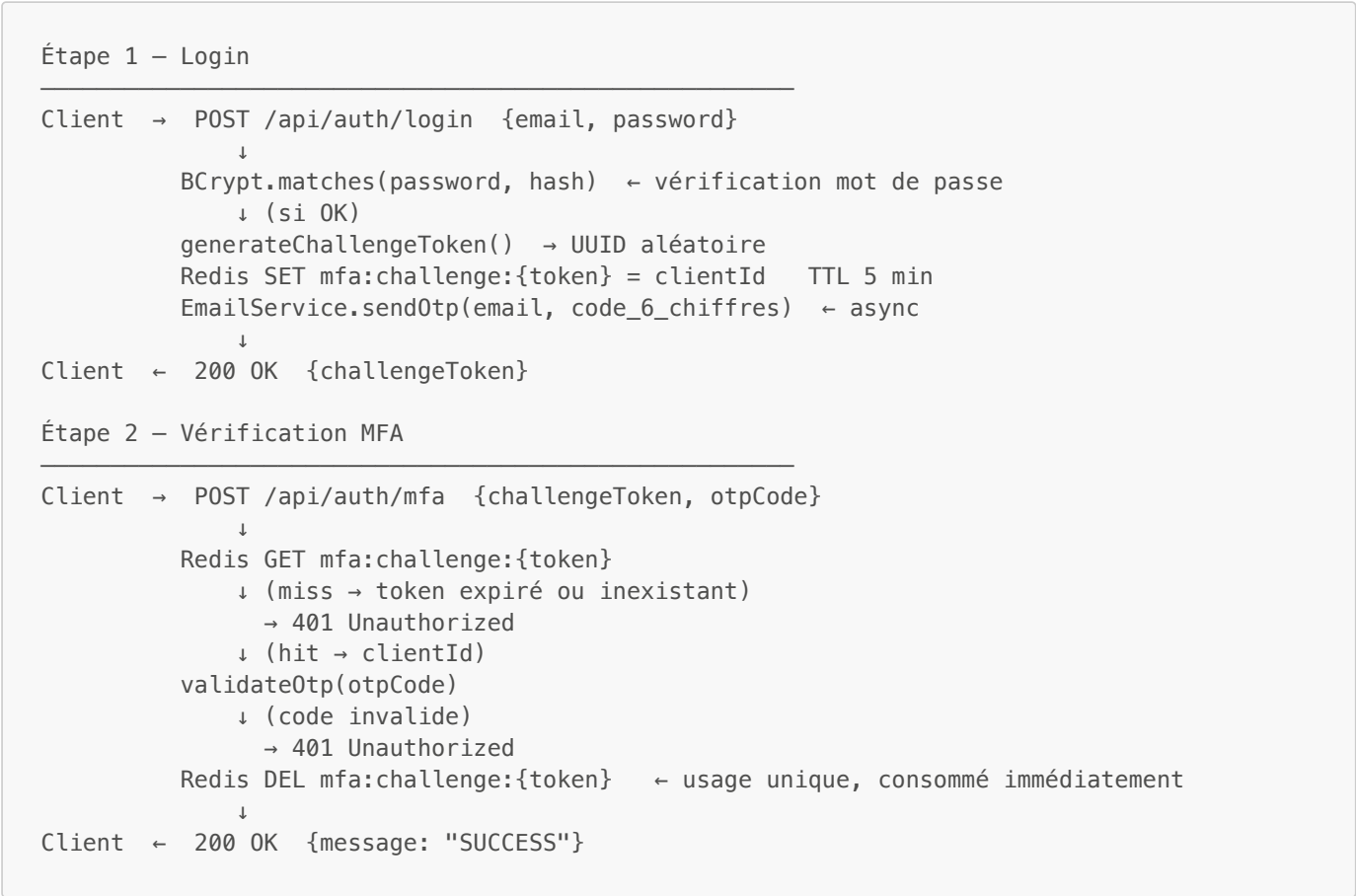
Choix du facteur de coût BCrypt :

Facteur	Temps/hash estimé	Commentaire
4	~5 ms	Trop rapide — insuffisant contre les attaques dictionnaire GPU
8	~25 ms	Retenu — acceptable à 50 VUs, ralentit significativement un attaquant
12	~250 ms	Cause une dégradation mesurable à 50+ VUs sous charge

Le smoke test confirme que BCrypt 8 n'impacte pas la latence perçue : p95 du login = 77 ms à 50 VUs. Le temps de hachage est masqué par le round-trip réseau et le débit MailHog.

4. Flux MFA en deux étapes

Le système implémente une **authentification à deux facteurs sans session HTTP** basée sur un token de challenge temporaire stocké dans Redis.



Propriétés de sécurité :

Propriété	Mécanisme
Expiration automatique	Redis TTL 5 minutes — aucune tâche planifiée de nettoyage
Usage unique	DEL immédiat après validation — replay impossible
Pas de session	Aucun cookie, aucun HttpSession — SessionCreationPolicy.STATELESS
Isolation par VU k6	La recherche MailHog filtre par challengeToken pour éviter les interférences entre VUs concurrents

5. Spring Security stateless

Chaque service configure Spring Security avec @Configuration + SecurityFilterChain :

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .sessionManagement(sm ->
            sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .csrf(AbstractHttpConfigurer::disable)
        .cors(cors -> cors.configurationSource(corsConfigurationSource()))
        .authorizeHttpRequests(auth -> auth.anyRequest().permitAll());
    return http.build();
}
```

Pourquoi pas de JWT ? Dans ce contexte académique, les appels entre services restent sur le réseau Docker interne (non exposé). L'absence de JWT est documentée comme dette technique dans [arc42.md §11](#) — en production, il faudrait intégrer Keycloak ou émettre des JWT signés après l'étape MFA.

6. Filtrage des en-têtes par KrakenD

KrakenD ne transmet que les en-têtes explicitement déclarés dans `krakend.json`. Tous les autres sont bloqués par défaut (header stripping).

En-têtes autorisés par service :

Header	identity-service	account-service	payment-service
Content-Type	✓	✓	✓
Authorization	✓	✓	✓
Idempotency-Key	×	×	✓
X-Forwarded-For	×	×	×

Le header `Idempotency-Key` est transmis **uniquement** vers `payment-service` pour l'endpoint `POST /api/transactions`. Sur tous les autres backends, KrakenD le supprime.

7. Gestion des erreurs normalisée

Le module `common` (dépendance Maven partagée) contient `GlobalExceptionHandler` (`@RestControllerAdvice`) qui intercepte toutes les exceptions avant qu'elles n'atteignent le client.

Format de réponse d'erreur `ErrorResponse` :

```
{
  "status": 422,
  "error": "Unprocessable Entity",
  "message": "Insufficient funds on account 1596875651",
  "timestamp": "2026-03-08T14:23:10.452Z"
}
```

Mapping exceptions → codes HTTP :

Exception	Code	Exemple de message
<code>MethodArgumentNotValidException</code>	400	"firstName must not be blank"
<code>ClientNotFoundException</code>	404	"Client not found: {uuid}"
<code>AccountNotFoundException</code>	404	"Account not found: {accountNumber}"
<code>PaymentNotFoundException</code>	404	"Transaction not found: {uuid}"
<code>BadCredentialsException</code>	401	"Invalid credentials"
<code>DuplicateEmailException</code>	409	"Email already registered"
<code>DuplicateNasException</code>	409	"NAS already registered"
<code>InsufficientFundsException</code>	422	"Insufficient funds on account {n}"
<code>Exception</code> (catch-all)	500	"Internal error"

Le smoke test valide **7 cas d'erreur** : `bad_password` → 401, `duplicate_email` → 409, `unknown_client` → 404, `overdraft` → 422, `bad_account` → 404, `bad_otp` → 401, `missing_idempotency_key` → 400.

8. Stratégie de versionnage

Plutôt qu'un segment `/v1/` générique, chaque service utilise son propre préfixe de namespace dans tous ses chemins internes :

- `identity-service` → `/identityservice/...`
- `account-service` → `/accountservice/...`
- `payment-service` → `/paymentservice/...`

Ce préfixe sert à deux fins simultanément :

1. **Versiionnage implicite** — un futur `identity-service` v2 pourrait exposer `/identityservice-v2/...` et coexister avec l'ancien.
2. **Routage KrakenD déterministe** — le gateway n'a pas besoin de connaître l'hôte+port de chaque service à partir du chemin ; il suffit de mapper le préfixe public `/api/...` vers le préfixe interne `/xxxservice/...`.

Les endpoints internes (`/debit`, `/credit`, `/identityservice/clients/{id}`) ne sont pas exposés via KrakenD — ils ne sont accessibles que depuis le réseau Docker interne.

9. CORS

CORS est activé sur les trois services pour permettre les appels depuis le frontend React (Vite, port 5173) et Swagger UI :

```
@Bean
CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOriginPatterns(List.of("*"));
    config.setAllowedMethods(List.of("GET", "POST", "PATCH", "PUT", "DELETE", "OPTIONS"));
    config.setAllowedHeaders(List.of("*"));
    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", config);
    return source;
}
```

Les pré-vols (`OPTIONS`) sont répondus directement par Spring Security sans atteindre les controllers.